Java Level 11 — DSA & Problem Solving

Part 4 — Heap / Priority Queue

Overview

This part focuses on recognizing problems where we repeatedly need the **smallest, largest, or Top K elements**.

Topics covered:

- Heap / Priority Queue fundamentals
- Min Heap
- Max Heap
- Java `PriorityQueue`
- Kth Largest Element
- Top K Frequent Elements
- Merge K Sorted Lists
- K Closest Points to Origin
- Top-K Heap pattern
- Heap recognition rules

---

1. What is a Heap?

A **Heap** is a data structure optimized for repeatedly accessing the smallest or largest element.

In Java, we normally use:

```
PriorityQueue
```

rather than implementing a heap ourselves.

Two important types:

```
Min Heap
→ smallest element at root

Max Heap
→ largest element at root
```

2. Java PriorityQueue

Min Heap

Java's `PriorityQueue` is a Min Heap by default.

```
PriorityQueue<Integer> minHeap =
        new PriorityQueue<>();
```

Example:

```
minHeap.offer(5);
minHeap.offer(2);
minHeap.offer(8);

System.out.println(minHeap.peek());
```

Output:

```
2
```

Max Heap

Use a reverse comparator:

```
PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>(
                Comparator.reverseOrder()
        );
```

Example:

```
maxHeap.offer(5);
maxHeap.offer(2);
maxHeap.offer(8);
```

```
System.out.println(maxHeap.peek());
```

Output:

```
8
```

---

3. Important Heap Operations

```
heap.offer(value);
```

Adds an element.

```
heap.peek();
```

Returns the root without removing it.

```
heap.poll();
```

Removes and returns the root.

Complexities:

Operation	Complexity
`peek()`	O(1)
`offer()`	O(log n)
`poll()`	O(log n)

---

4. When Should I Think About a Heap?

Heap is a strong candidate when the problem contains phrases like:

```
K largest
K smallest
```

```
Kth largest
Kth smallest
Top K
Closest K
Most frequent K
Highest priority
Lowest priority
Repeatedly get minimum
Repeatedly get maximum
Merge multiple sorted sources
```

A very important distinction:

```
Need all elements sorted
→ Sorting may be simpler

Need only K best elements
→ Heap can avoid sorting everything
```

Sorting:

```
O(n log n)
```

Maintaining a heap of size `k`:

```
O(n log k)
```

This becomes useful when:

```
k << n
```

---

5. The Most Important Top-K Rule

This is the main Heap rule to remember.

```
Want K LARGEST
→ MIN Heap of size K
```

```
Want K SMALLEST
→ MAX Heap of size K
```

This can initially feel backwards.

Instead of memorizing it blindly, remember:

> The root should be the **worst candidate among the K elements we currently want to keep**.

---

6. Why K Largest Uses a Min Heap

Suppose:

```
k = 3
```

Current Top 3 largest:

```
10
20
30
```

The worst candidate among them is:

```
10
```

If a new number comes:

```
40
```

we want to remove:

```
10
```

Therefore the smallest winner should be easy to remove.

That means:

```
MIN Heap
```

Pattern:

```
for (int num : nums) {

    minHeap.offer(num);

    if (minHeap.size() > k) {
        minHeap.poll();
    }
}
```

At the end:

```
minHeap.peek()
```

is the Kth largest element.

---

7. Why K Smallest Uses a Max Heap

Suppose we want the three smallest values.

Current winners:

```
2
4
7
```

The worst candidate is:

```
7
```

If a new value arrives:

```
3
```

we want:

```
2
3
4
```

So `7` should be easy to remove.

Therefore the largest winner should be at the root:

```
MAX Heap
```

---

8. Problem — Kth Largest Element

Problem

Given:

```
nums = [3,2,1,5,6,4]
k = 2
```

Return:

```
5
```

because the sorted order is:

```
6, 5, 4, 3, 2, 1
   ↑
2nd largest
```

Pattern

We want:

```
K largest elements
```

Therefore:

```
Min Heap of size K
```

Solution

```
public static int findKthLargest(
        int[] nums,
        int k
) {

    PriorityQueue<Integer> minHeap =
            new PriorityQueue<>();

    for (int num : nums) {

        minHeap.offer(num);

        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }

    return minHeap.peek();
}
```

Why It Works

For:

```
nums = [3,2,1,5,6,4]
k = 2
```

we continuously maintain only the two largest elements seen so far.

Eventually:

```
Heap contains:
```

```
5
6
```

The smallest of those two is:

```
5
```

which is the second largest overall.

Complexity

```
Time  → O(n log k)
Space → O(k)
```

---

9. Problem — Top K Frequent Elements

Problem

Given:

```
nums = [1,1,1,2,2,3]
k = 2
```

Return:

```
[1,2]
```

because:

```
1 → frequency 3
2 → frequency 2
3 → frequency 1
```

---

10. Pattern — HashMap + Heap

This problem combines two patterns:

```
HashMap
→ count frequencies

Heap
→ maintain Top K
```

First:

```
Map<Integer, Integer> frequency =
        new HashMap<>();

for (int num : nums) {
    frequency.put(
            num,
            frequency.getOrDefault(num, 0) + 1
    );
}
```

Now:

```
1 → 3
2 → 2
3 → 1
```

---

11. Heap Based on Frequency

We don't care about the numerical value.

We care about:

```
frequency
```

So our comparator should compare frequencies.

```
PriorityQueue<Integer> minHeap =
        new PriorityQueue<>(
                Comparator.comparingInt(frequency::get)
        );
```

This means:

```
lowest frequency
→ root
```

---

12. Top K Frequent Solution

```
public static int[] topKFrequent(
        int[] nums,
        int k
) {

    Map<Integer, Integer> frequency =
            new HashMap<>();

    for (int num : nums) {
        frequency.put(
                num,
                frequency.getOrDefault(num, 0) + 1
        );
    }

    PriorityQueue<Integer> minHeap =
            new PriorityQueue<>(
                    Comparator.comparingInt(frequency::get)
            );

    for (int num : frequency.keySet()) {

        minHeap.offer(num);

        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }

    int[] result = new int[k];
```

```
    for (int i = 0; i < k; i++) {
        result[i] = minHeap.poll();
    }

    return result;
}
```

---

13. Important Mistake — Heap Size Changes During Poll

An incorrect extraction attempt was:

```
for (int i = 0; i < minHeap.size(); i++) {
    result[i] = minHeap.poll();
}
```

The problem:

```
minHeap.size()
```

changes every time `poll()` is called.

Example:

```
Initial size = 3

i = 0
poll()
size = 2

i = 1
poll()
size = 1

condition:
1 < 1 → false
```

The loop ends too early.

Instead:

```
for (int i = 0; i < k; i++) {
    result[i] = minHeap.poll();
}
```

Or:

```
int i = 0;

while (!minHeap.isEmpty()) {
    result[i++] = minHeap.poll();
}
```

---

14. Important Comparator Lesson

With:

```
PriorityQueue<Integer> minHeap =
        new PriorityQueue<>(
                Comparator.comparingInt(frequency::get)
        );
```

calling:

```
minHeap.poll();
```

does NOT remove:

```
smallest number
```

It removes:

```
number having the smallest frequency
```

The comparator defines what:

```
smallest
```

means to the heap.

This is an important PriorityQueue concept.

---

15. Top K Frequent Complexity

Let:

```
n = total numbers
m = unique numbers
```

Frequency counting:

```
O(n)
```

Heap processing:

```
O(m log k)
```

Total:

```
O(n + m log k)
```

Since:

```
m <= n
```

we can commonly describe the upper bound as:

```
O(n log k)
```

Space:

```
O(m + k)
```

---

16. Problem — Merge K Sorted Lists

🟡 **REVISIT**

This problem was covered, but the logic was not completely comfortable yet.

We intentionally moved forward and will revisit it later.

Problem

Given:

```
List 1: 1 → 4 → 5
List 2: 1 → 3 → 4
List 3: 2 → 6
```

Return:

```
1 → 1 → 2 → 3 → 4 → 4 → 5 → 6
```

---

17. Core Merge K Idea

Each list is already sorted.

At any moment, we only need to know:

> Which list currently has the smallest available node?

Example:

```
List 1: [1] → 4 → 5
List 2: [1] → 3 → 4
List 3: [2] → 6
```

Initial candidates:

```
1
1
2
```

Put those into a:

```
Min Heap
```

Then repeatedly:

```
Get smallest candidate
        ↓
Add it to result
        ↓
Take next node from that same list
        ↓
Put that node into heap
        ↓
Repeat
```

---

18. Merge K Sorted Lists Solution

```
public static ListNode mergeKLists(
        ListNode[] lists
) {

    PriorityQueue<ListNode> minHeap =
            new PriorityQueue<>(
                    Comparator.comparingInt(
                            node -> node.val
                    )
            );

    for (ListNode list : lists) {

        if (list != null) {
            minHeap.offer(list);
        }
    }
```

```
    ListNode dummy = new ListNode(0);
    ListNode current = dummy;

    while (!minHeap.isEmpty()) {

        ListNode smallest = minHeap.poll();

        current.next = smallest;
        current = current.next;

        if (smallest.next != null) {
            minHeap.offer(smallest.next);
        }
    }

    return dummy.next;
}
```

---

19. Merge K Mental Model

The key thing to remember for revision:

```
Multiple sorted sources
+
Repeatedly need smallest current item

→ Min Heap
```

The heap contains at most:

```
one current candidate from each list
```

If there are `K` lists:

```
Heap size <= K
```

Complexity

Let:

```
N = total nodes
K = number of lists
```

Each node eventually enters and leaves the heap.

Heap operation:

```
O(log K)
```

Therefore:

```
Time  → O(N log K)
Space → O(K)
```

Status

```
🟡 Revisit later
```

Do not treat this problem as mastered yet.

---

20. Problem — K Closest Points to Origin

Problem

Given:

```
points = [[1,3], [-2,2]]
k = 1
```

Return the point closest to:

```
(0,0)
```

---

21. Distance Formula

Normally:

```
distance = √(x² + y²)
```

But we don't need the square root.

For comparison:

```
√5 < √10
```

is equivalent to:

```
5 < 10
```

So use:

```
int distance =
        x * x + y * y;
```

Example:

```
[1,3]

1² + 3²
= 10
```

and:

```
[-2,2]

(-2)² + 2²
= 8
```

Therefore:

```
[-2,2]
```

is closer.

---

22. Which Heap for K Closest?

We want:

```
K smallest distances
```

Therefore:

```
MAX Heap of size K
```

Remember:

```
K smallest
→ Max Heap
```

because the largest distance among our current winners is the:

```
worst candidate
```

and should be easy to remove.

---

23. K Closest Solution

```
public static int[][] kClosest(
        int[][] points,
        int k
) {

    PriorityQueue<int[]> maxHeap =
            new PriorityQueue<>(
                    (a, b) -> {
```

```
                        int distA =
                                a[0] * a[0]
                                + a[1] * a[1];

                        int distB =
                                b[0] * b[0]
                                + b[1] * b[1];

                        return Integer.compare(
                                distB,
                                distA
                        );
                    }
            );

    for (int[] point : points) {

        maxHeap.offer(point);

        if (maxHeap.size() > k) {
            maxHeap.poll();
        }
    }

    int[][] result =
            new int[k][2];

    for (int i = 0; i < k; i++) {
        result[i] = maxHeap.poll();
    }

    return result;
}
```

---

24. Important Mistake — Wrong Heap Direction

An initial comparator was:

```
return Integer.compare(
        distA,
        distB
);
```

This creates:

```
Min Heap
```

The closest point would be at the root.

But then our algorithm does:

```
if (maxHeap.size() > k) {
    maxHeap.poll();
}
```

That would remove the:

```
closest point
```

which is exactly what we want to keep.

Instead:

```
return Integer.compare(
        distB,
        distA
);
```

creates:

```
Max Heap
```

Now the farthest point among the current candidates is removed.

---

25. The Universal Top-K Pattern

This is the main pattern from this entire chapter.

Suppose we need:

```
Top K items
```

Step 1:

Define what makes an item:

```
better
```

Examples:

```
larger number
smaller number
higher frequency
smaller distance
higher score
```

Step 2:

Maintain only:

```
K candidates
```

Step 3:

Make the heap root represent:

```
the WORST candidate
among the current winners
```

Step 4:

Whenever:

```
heap.size() > k
```

remove:

```
heap.poll()
```

26. Heap Direction Cheat Sheet

Want K Largest

Example:

```
Top 3 salaries
Top 5 scores
Kth largest number
```

Use:

```
MIN HEAP
```

Why?

```
Smallest winner
= worst winner
```

Want K Smallest

Example:

```
K closest points
K smallest numbers
K lowest costs
```

Use:

```
MAX HEAP
```

Why?

```
Largest winner
= worst winner
```

27. Problem Recognition Cheat Sheet

Problem clue	Pattern
Kth largest	Min Heap size K
K largest	Min Heap size K
Kth smallest	Max Heap size K
K smallest	Max Heap size K
Top K frequent	HashMap + Min Heap
K closest	Max Heap based on distance
Repeated smallest	Min Heap
Repeated largest	Max Heap
Multiple sorted sources	Min Heap
Merge K sorted lists	Min Heap
Priority processing	PriorityQueue

28. Common Mistakes

Mistake 1 — Confusing Min Heap and Max Heap

Don't memorize:

```
largest → max heap
smallest → min heap
```

That is often wrong for Top-K problems.

Instead ask:

> Which candidate should be removed when I have more than K?

The root should be that candidate.

Mistake 2 — Sorting Everything

For:

```
Kth largest
```

you could sort:

```
O(n log n)
```

But a heap of size `k` gives:

```
O(n log k)
```

When `k` is small, the heap is more efficient.

Mistake 3 — Forgetting Heap Size Limit

The Top-K pattern usually contains:

```
if (heap.size() > k) {
    heap.poll();
}
```

This is what keeps the heap at:

```
O(k)
```

space.

Mistake 4 — Comparator Direction

These are different:

```
Integer.compare(a, b)
```

and:

```
Integer.compare(b, a)
```

The first behaves like:

```
ascending / Min Heap
```

The second behaves like:

```
descending / Max Heap
```

Mistake 5 — Forgetting What Comparator Represents

A heap doesn't necessarily compare the actual number.

For example:

```
Comparator.comparingInt(
        frequency::get
)
```

means:

```
Priority is determined by frequency
```

not the numerical value itself.

Mistake 6 — Using Heap Size in a Shrinking Loop

Avoid:

```
for (int i = 0; i < heap.size(); i++) {
    heap.poll();
}
```

because:

```
heap.size()
```

changes during the loop.

Use a known size:

```
for (int i = 0; i < k; i++) {
    heap.poll();
}
```

or:

```
while (!heap.isEmpty()) {
    heap.poll();
}
```

---

29. Complexity Cheat Sheet

Standard Heap

```
peek  → O(1)
offer → O(log n)
poll  → O(log n)
```

Top K Pattern

Heap size is limited to `k`:

```
offer/poll → O(log k)
```

Processing `n` elements:

```
O(n log k)
```

Space:

```
O(k)
```

Merge K Sorted Lists

```
N = total nodes
K = number of lists

Time  → O(N log K)
Space → O(K)
```

---

30. Heap Mental Models

Model 1 — Top K

```
Process every item

        ↓

Add candidate

        ↓

More than K?

        ↓ yes

Remove worst candidate
```

Model 2 — Root = Worst Winner

For Top-K problems:

```
Heap contains current winners.

Heap root should be:
```

```
the easiest winner to replace.
```

This single idea helps determine:

```
Min Heap vs Max Heap
```

Model 3 — Multiple Sorted Sources

```
Sorted Source 1 ─┐
Sorted Source 2 ─┼→ Heap → smallest
Sorted Source 3 ─┘
```

After taking one element:

```
add the next element
from the same source
```

This is the key idea behind:

```
Merge K Sorted Lists
```

🟡 Revisit this pattern later.

---

31. Interview Explanation

If asked why you used a heap for a Top-K problem:

> We only need the K best elements rather than a fully sorted collection. I maintain a heap of size K where the root represents the worst candidate among the current winners. Whenever the heap grows beyond K, I remove the root. This reduces the complexity from sorting all N elements in O(N log N) to O(N log K).

---

32. Revision Checklist

Before interviews, you should immediately recognize:

```
Kth largest
→ Min Heap size K
```

```
K largest
→ Min Heap size K
```

```
K smallest
→ Max Heap size K
```

```
Top K frequent
→ HashMap + Heap
```

```
K closest
→ Max Heap based on distance
```

```
Multiple sorted sources
→ Min Heap
```

And always remember:

```
ROOT = WORST CURRENT WINNER
```

---

33. Practice Status

Topic	Status
PriorityQueue fundamentals	✅
Min Heap	✅
Max Heap	✅
Kth Largest	✅
Top K Frequent	✅
K Closest Points	✅
Merge K Sorted Lists	🟡 Revisit

Topic	Status
Top-K recognition	✅

---

34. Level 11 Progress

Part 1 — Core Patterns

- ✅ Big-O
- ✅ Arrays & Hashing
- ✅ Two Pointers
- ✅ Sliding Window
- ✅ Prefix Sum
- ✅ Stack
- ✅ Monotonic Stack
- ✅ Queue / Deque Basics
- 🟡 Monotonic Deque / Sliding Window Maximum — Revisit

Part 2 — Binary Search & Linked Lists

- ✅ Binary Search
- ✅ Search Insert Position
- ✅ First / Last Occurrence
- ✅ Search Rotated Sorted Array
- ✅ Binary Search on Answer
- ✅ Reverse Linked List
- ✅ Slow & Fast Pointers
- ✅ Middle of Linked List
- ✅ Linked List Cycle
- ✅ Dummy Node
- ✅ Merge Two Sorted Lists
- 🟡 Remove Nth Node From End — implementation pending

Part 3 — Trees & BST

- ✅ Binary Tree Fundamentals
- ✅ DFS
- ✅ Preorder / Inorder / Postorder
- ✅ Maximum Depth
- ✅ Same Tree
- ✅ Invert Binary Tree
- ✅ BST Fundamentals
- ✅ Search BST
- ✅ Validate BST

- ✅ Lowest Common Ancestor
- ✅ BFS / Level Order
- ✅ DFS vs BFS

Part 4 — Heap / Priority Queue

- ✅ Heap fundamentals
- ✅ Min Heap / Max Heap
- ✅ Kth Largest
- ✅ Top K Frequent
- ✅ K Closest Points
- 🟡 Merge K Sorted Lists — Revisit

Next — Part 5

Graphs

Next topics:

```
Graph Representation
        ↓
Adjacency List
        ↓
Visited Set
        ↓
Graph DFS
        ↓
Graph BFS
        ↓
Connected Components
        ↓
Cycle Detection
        ↓
Topological Sort
        ↓
Union Find
```

The important connection going into Part 5:

```
Tree DFS
    ↓
Graph DFS + visited
```

```
Tree BFS
    ↓
Graph BFS + visited
```

Graphs mainly introduce:

```
cycles
multiple paths
disconnected components
visited tracking
```